Docker, Compose, Profiles, Flyway (Days 66-70)

First, resolve a mismatch: the roadmap says "PostgreSQL" for Days 67 and 70, but this project has used **MySQL** from Day 1 (`mysql-connector-j` in `pom.xml`, `jdbc:mysql://...` everywhere). Switching databases this late is real risk for no real benefit — recommendation: keep MySQL, treat every "PostgreSQL" mention below as "MySQL" instead. Don't migrate.

1. Dockerfile (Day 66)

```
# --- build stage ---
FROM eclipse-temurin:23-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw clean package -DskipTests

# --- run stage ---
FROM eclipse-temurin:23-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Test: `docker build -t banking-app .` then `docker run -p 8080:8080 -e DB_PASSWORD=... -e JWT_SECRET=... banking-app` — confirm it starts (won't connect to MySQL yet until Compose wires networking, next step).

2. docker-compose.yml — the full stack (Day 67)

```yaml
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
      MYSQL_DATABASE: banking_db
    ports: ["3306:3306"]

  redis:
    image: redis:7
    ports: ["6379:6379"]

  kafka:
    image: apache/kafka:3.8.0
    ports: ["9092:9092"]
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  banking-app:
    build: .
    depends_on: [mysql, redis, kafka]
    ports: ["8080:8080"]
    environment:
      DB_PASSWORD: ${DB_PASSWORD}
      JWT_SECRET: ${JWT_SECRET}
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/banking_db
      SPRING_DATA_REDIS_HOST: redis
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092

  notification-service:
    build: ../notification-service
    depends_on: [mysql, kafka]
    ports: ["8081:8081"]
    environment:
      DB_PASSWORD: ${DB_PASSWORD}
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/notification_db
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
```

Note `kafka:9092` not `localhost:9092` inside Compose — containers reach each other by service name, not `localhost`.

Test: `docker compose up -d`, then `docker compose ps` — all 5 services `Up`. Hit `http://localhost:8080/actuator/health`.

3. Profiles (Day 68)

`application-dev.properties`:

```
spring.jpa.show-sql=true
logging.level.org.example.enterprisedigitalbankingsystem=DEBUG
```

`application-prod.properties`:

```
spring.jpa.show-sql=false
spring.jpa.hibernate.ddl-auto=validate
logging.level.org.example.enterprisedigitalbankingsystem=INFO
```

Run with `--spring.profiles.active=dev` locally, `prod` in the docker-compose `environment:` block (`SPRING_PROFILES_ACTIVE: prod`).

4. Flyway (Day 69)

```
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-mysql</artifactId>
</dependency>
```

Switch `spring.jpa.hibernate.ddl-auto=update` → `validate` (Flyway owns the schema now, Hibernate just checks it matches). Create `src/main/resources/db/migration/V1__baseline.sql` — the simplest correct baseline is to let Hibernate generate the current schema once (temporarily set `ddl-auto=update`, start the app, then use `mysqldump --no-data banking_db > V1__baseline.sql` to capture exactly what exists), then switch to `validate` for good.

Test: fresh database, start the app — Flyway should run `V1__...` automatically and log `Successfully applied 1 migration`.

5. Indexes (Day 70)

Most of what you need already exists as side effects of `unique = true` (MySQL auto-indexes unique columns): `accounts.account_number`, `users.email`, `beneficiaries` lookup, etc. Add explicit indexes for columns queried often but not unique — e.g. `transactions.source_account_id` and `destination_account_id` (used by the history query):

```
CREATE INDEX idx_txn_source ON transactions(source_account_id);
CREATE INDEX idx_txn_dest ON transactions(destination_account_id);
```

Add as `V2__indexes.sql` once Flyway is in place.